ML-Assisted Detection of IoT Bot-net DDoS Attacks: A Comparative Study

Kananelo Chabeli
Department of Electrical Engineering
University of Cape Town
Cape Town, South Africa
chbkan001@myuct.ac.za

***Abstract*—Software Defined Networks (SDN) have greatly improved network management, programmability, and flexibility by separating the control plane from the data plane. However, this separation has also created new opportunities for malicious activity within the network. One major threat is Distributed Denial of Service (DDoS) attacks, which continue to attract significant research attention. Recent studies have increasingly explored the use of machine learning techniques to detect and mitigate such attacks.**

Among these approaches, ensemble learning has shown strong potential for network traffic classification. By combining multiple base models, it often achieves better performance than individual models. Despite this promise, the area remains insufficiently explored in the current literature.

This study presents a detailed comparative analysis of six widely used tree-based ensemble learning methods, namely Random Forest, Extra Trees, XGBoost, LightGBM, CatBoost, and AdaBoost. These models are evaluated using both a custom dataset and the CICIoT-DIAD2024 dataset. The results show that XGBoost outperformed the other models on the CICIoT-DIAD2024 dataset, achieving an accuracy of 96.35%, while LightGBM achieved a notably higher accuracy of 99.76% on custom dataset.

***Index Terms*—SDN,DDoS, Ensemble Learning, IoT.**

I. Introduction

In traditional networks, packet forwarding (data plane) and routing computation (control plane) were tightly coupled within each device, forming a distributed architecture [1]. This design made managing large networks difficult. Software Defined Networking (SDN) changes this by separating the application, control, and data planes [1]–[3], which makes networks more flexible and easier to manage. However, this architectural shift introduces a new risk. The SDN controller, which now acts as the brain of the network, becomes a high-value target for attackers. If an adversary can overwhelm or compromise it, the consequences ripple across the entire network [4], [5]. One of the most effective ways to do this is through a Distributed Denial of Service (DDoS) attack, where a large number of machines flood the network with traffic to exhaust its resources and render it unavailable [1], [2].

The rise of the Internet of Things (IoT) has made this problem considerably worse. IoT devices such as smart sensors, cameras, thermostats, and similar equipment are deployed in enormous numbers, often with minimal security protections. They are cheap to manufacture, computationally constrained, and rarely updated after deployment, which makes them easy targets for compromise [6]. Once infected, they can be conscripted into large networks of compromised machines called *botnets*, which attackers use to coordinate and amplify DDoS campaigns at a scale that would be difficult to achieve otherwise. The convergence of SDN and IoT, while technically powerful, therefore creates an environment that is particularly vulnerable to large-scale, botnet-driven DDoS attacks.

Detecting these attacks reliably and efficiently is an active area of research. Early approaches relied on statistical methods, tracking metrics like entropy to detect unusual deviations in traffic behaviour [4], [7]. While lightweight, these methods struggle to cope with the growing sophistication of attack traffic, which increasingly mimics normal behaviour. This has led researchers to explore machine learning (ML) as a more adaptive alternative. ML-based systems can learn to recognise complex patterns in network traffic without needing hand-crafted rules, making them better suited to evolving threats [8].

Within the ML landscape, *ensemble learning* has attracted particular attention. Rather than relying on a single model, ensemble methods aggregate the outputs of multiple base learners, often achieving better accuracy and robustness than any individual model [6], [9]. Among the different families of ensemble methods, tree-based ensembles, such as Random Forest, Gradient Boosting variants, and their modern implementations, have proven especially effective on structured network traffic data. They are fast to train, relatively interpretable, and capable of handling the high-dimensional, imbalanced datasets that are common in network security.

Despite this promise, the existing literature has not yet provided a thorough, head-to-head comparison of the major tree-based ensemble algorithms in the context of IoT botnet DDoS detection. Several studies have applied one or two ensemble methods as part of a broader comparison that includes non-tree-based classifiers [6], [9], [10], and others have focused on a specific model family without examining the full landscape [11]. Crucially, most prior comparisons rely on older or less realistic datasets that do not fully capture the traffic patterns of modern IoT-based botnet attacks. This leaves an important question unanswered: *among the widely used tree-based ensemble models, which one is best suited for detecting IoT botnet DDoS attacks, and does that conclusion hold across different network environments?*

This paper addresses this gap directly. A systematic compar-

ative study of six prominent tree-based ensemble classifiers, Random Forest (RF), Extra Trees (ET), AdaBoost, XGBoost, LightGBM, and CatBoost , is conducted and these models are evaluated under consistent experimental conditions. The contributions of this study are twofold:

1) All six models are benchmarked on the CICIoT-DIAD2024 dataset [12], a recently published, realistic IoT traffic dataset that captures a wide range of DDoS attack types.
2) To assess whether these findings generalise beyond a single benchmark,a custom IoT network topology is created and new dataset using the Mininet emulator and the POX SDN controller is generated. Several common IoT bot-net DDoS attack types in a controlled SDN-IoT environment are simulated, generating traffic that closely mirrors the structure and characteristics of CICIoT-DIAD2024. All six models are then re-evaluated on this dataset, providing an additional layer of validation for drawing more informed conclusions conclusions.

The remainder of this paper is organised as follows.Section II provides background on software defined networking architecture the ensemble learning methods studied. In, Section III a brief literature review covering both machine-learning and statistical-based detection algorithms is presented. Section IV describes the datasets, experimental set-up, and evaluation methodology. Next, Section V presents and discusses the results while Section VI concludes the paper and outlines directions for future work.

II. Background

A. Software Defined Networking

Software-Defined Networking (SDN) introduces a paradigm shift in networking by decoupling the control plane from the data plane. This subsection provides a high-level overview of the history and architecture of SDN. The SDN architecture is typically described using three layers: the Application layer, the Control layer, and the Data Plane layer. The Application layer communicates with the Control layer via northbound APIs, while the Control layer communicates with the Data Plane via southbound APIs. These interactions are illustrated in Fig 1. Although not shown in the figure, SDN controllers may also communicate with one another through eastbound and westbound APIs within the control plane.

SDN did not emerge from a single design point, but rather evolved through a series of ideas and innovations. Two particularly compelling precursors are the Routing Control Platform (RCP) [13] and the 4D Approach to Network Control and Management [14]. The RCP represents a logically centralized system responsible for computing routing decisions based on a global network view, while traditional routers are relegated to simple packet forwarding roles. This separation improved policy control, scalability, and network manageability, and established key principles that later influenced SDN, particularly centralized control and data plane abstraction.

Building on this direction, the 4D architecture introduced a more structured decomposition of network control into four distinct planes: Decision, Dissemination, Discovery, and Data. The Decision plane computes network-wide policies and configurations, the Dissemination plane distributes control information and collects network state, the Discovery plane identifies network topology and elements, and the Data plane handles packet forwarding. This clean separation addressed the increasing complexity of traditional networks and reinforced the concept of centralized and modular control, further shaping the evolution of SDN.

The practical realization of SDN was enabled by the introduction of the OpenFlow protocol, which provided a standardized interface for achieving full separation between the control and data planes. According to Waseem et al. [3], the evolution of SDN can be categorized into three stages: (i) Active Networking (mid-1990s to early 2000s), (ii) separation of control and data planes (2001–2007), during which RCP and the 4D architecture were proposed, and (iii) the emergence of OpenFlow and Network Operating Systems (NOS) (2007–2010), which enabled fully functional SDN deployments.

SDN offers several advantages. Centralization provides a global view of the network, simplifying configuration and management. The decoupling of the control and data planes allows each to evolve independently, promoting flexibility and innovation. Furthermore, SDN can reduce costs compared to traditional networks, where tightly coupled control and data planes increase hardware complexity. However, SDN also introduces several challenges. Its architectural departure from traditional networking makes integration with legacy systems difficult. The separation of control and data planes introduces potential issues such as single points of failure, increased attack surfaces, and control plane latency. In multi-controller environments, additional challenges arise, including optimal controller placement to minimize controller-to-switch (CTS) and controller-to-controller (CTC) latencies.

In this study, Mininet and the POX SDN controller are used to implement an IoT-based network simulation, as described in the next section. Having reviewed SDN, the following subsection presents background information on the ensemble algorithms used in this study.

B. Ensemble Learning

Ensemble learning is a machine learning technique that improves predictive performance by combining the outputs of multiple individual models [9], [11]. This aggregation reduces overall prediction error by simultaneously lowering both variance and bias. Ensemble algorithms are broadly categorised into two groups, Bagging and Boosting, based on how they combine predictions from their base learners.

In Bagging ensemble models, multiple learners are trained simultaneously on different bootstrapped subsets of the dataset, and their predictions are aggregated to produce a final output. Key advantages of Bagging include reduced variance, since aggregating predictions from diverse base learners makes the ensemble less prone to overfitting, and faster training

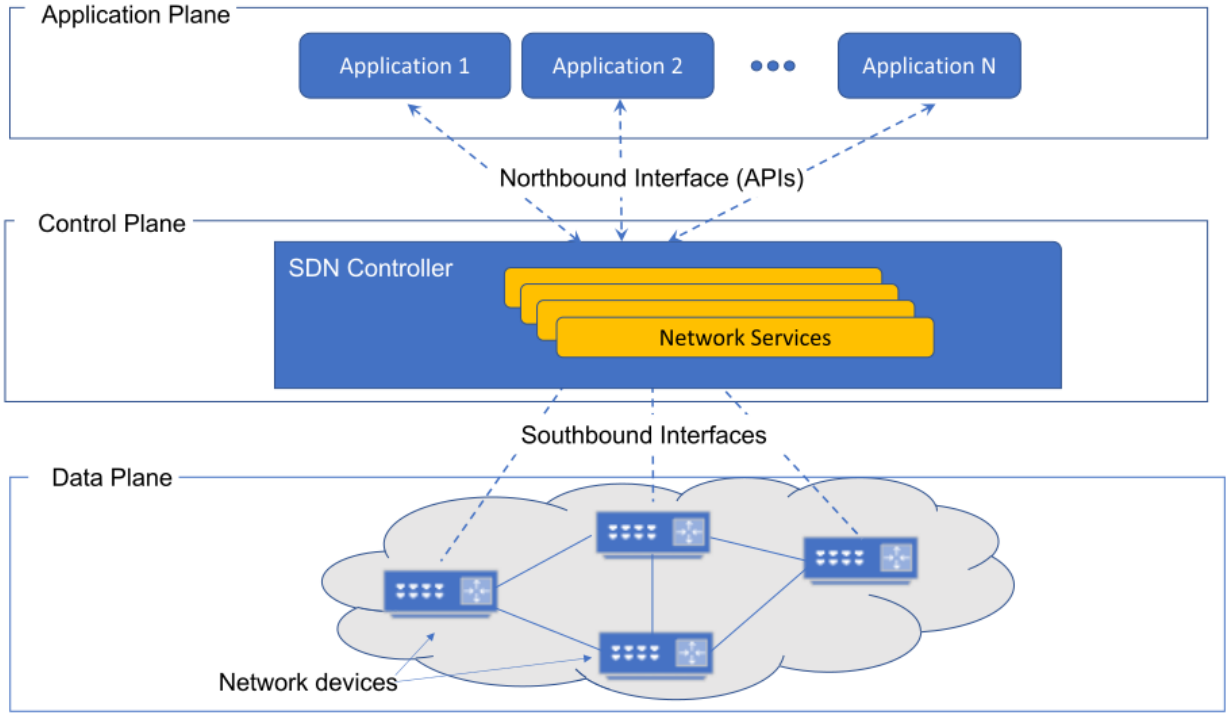

Fig. 1: Software Defined Networking Architecture [1]

speeds, owing to the parallelisable nature of the training process. However, Bagging is less effective at reducing bias. Bias is a systematic error that quantifies how far predictions deviate from true values, typically resulting from overly simplistic model assumptions. Because Bagging trains models in parallel and aggregates only their final predictions, it cannot correct for bias when the majority of base learners are themselves high-bias (i.e., underfitting) models.

Boosting ensemble models are designed to address this limitation. Rather than training base learners in parallel, Boosting trains them sequentially, where each successive model attempts to correct the errors made by the previous one. This iterative error-correction mechanism reduces both bias and variance, making boosting models less prone to both underfitting and overfitting. A further advantage of this approach is that it is highly effective even when the individual base learners are very weak (i.e., only slightly better than random guessing). However, because Boosting assigns greater weight to misclassified instances, it is sensitive to noise and outliers, which can distort the learning process. Furthermore, sequential training makes these models more computationally intensive than their Bagging counterparts.

In this study, six popular ensemble learning algorithms are evaluated for their performance in DDoS attack detection. This set comprises two Bagging algorithms, Random Forest and Extra Trees, and four Boosting algorithms, AdaBoost, eXtreme Gradient Boosting (XGBoost), CatBoost, and Light Gradient Boosting Machine (LightGBM).

a) Random Forest: Random Forest (RF) is a Bagging ensemble algorithm introduced by Breiman in 2001 [15]. It operates by constructing a large number of decision trees, each trained on a different bootstrapped sample of the training data. To further decorrelate the trees and reduce variance, RF introduces an additional layer of randomness during tree construction: at each node split, only a random subset of features is considered rather than all available features. The final prediction is determined by a majority vote (for classification) or average (for regression) across all trees in the ensemble.

The primary strength of RF is its strong resistance to overfitting. Because predictions are averaged across many decorrelated trees, no single tree can dominate the final output, leading to robust generalisation performance [15]. RF is also capable of handling high-dimensional datasets with large numbers of features, and it produces a natural measure of feature importance, which can be valuable for understanding which input variables most influence predictions. Additionally, RF requires minimal hyperparameter tuning, making it accessible and reliable even for practitioners new to machine learning. A further practical advantage is that the out-of-bag (OOB) error, computed using the data samples excluded from each bootstrap, can serve as a reliable internal estimate of generalisation error without requiring a separate validation set [15].

The main limitation of RF is that it can produce large models requiring substantial memory, particularly when a high number of trees is used. Moreover, because it inherits the Bagging paradigm, RF has limited ability to reduce bias; if

the underlying decision trees are individually high-bias, the ensemble will also suffer from underfitting. Finally, while RF provides feature importance scores, the ensemble nature of the model reduces interpretability compared to a single decision tree.

b) Extra Trees: Extra Trees (Extremely Randomised Trees) is a Bagging ensemble algorithm proposed by Geurts, Ernst, and Wehenkel in 2006 [16]. It is closely related to Random Forest but introduces an additional layer of randomisation during training. While RF selects the best split from a random subset of features at each node, Extra Trees randomly selects both the features *and* the split thresholds, choosing the best among these random candidates. This means that Extra Trees does not search for the optimal split point, but instead picks cut-points at random and selects the best among them. In the most extreme case, each tree structure is determined entirely independently of the output labels, producing totally randomised trees.

This heightened level of randomisation offers two key advantages. First, Extra Trees is computationally faster than Random Forest, since it eliminates the need to sort features or search for optimal split thresholds [16]. Second, the increased diversity among trees can further reduce variance, and studies have shown that Extra Trees achieves accuracy competitive with, and sometimes superior to, Random Forest across a range of classification and regression problems [16]. The algorithm is also reported to be particularly effective on noisy datasets, where overfitting to specific data points is a concern.

The trade-off for this additional randomisation is a potential increase in bias, since split thresholds are not optimised. In practice, this means that Extra Trees may perform slightly worse than Random Forest when the dataset is clean and the signal-to-noise ratio is high. Like RF, Extra Trees models can be large and memory-intensive, and the ensemble nature of the model reduces interpretability compared to a single decision tree.

c) AdaBoost: Adaptive Boosting (AdaBoost) is a Boosting ensemble algorithm introduced by Freund and Schapire in 1997 [17]. It was the first practical and widely adopted boosting algorithm and works by training a sequence of weak learners, typically shallow decision trees known as decision stumps (trees with a single split). What distinguishes AdaBoost from other ensemble methods is its adaptive sample weighting mechanism: after each round of training, samples that were misclassified are assigned higher weights, causing the next learner to focus more attention on the harder-to-classify examples. The final prediction is a weighted majority vote of all weak learners, where more accurate learners are assigned greater influence.

The principal strength of AdaBoost is its ability to convert a collection of weak learners into a strong classifier with high predictive accuracy [17]. It is relatively simple to implement, requires few hyperparameters to tune (primarily the number of boosting rounds), and works well across a wide range of classification tasks. Because it iteratively focuses on difficult cases, AdaBoost is highly effective at reducing both bias and variance, and it has been shown to be relatively resistant to overfitting in many practical settings [18].

However, AdaBoost has a significant sensitivity to noisy data and outliers. Because mislabelled or anomalous samples are repeatedly assigned high weights, the algorithm can over-emphasise these erroneous instances, leading to poor generalisation on noisy datasets [18]. Furthermore, since training is sequential, AdaBoost can be computationally slower than Bagging methods when the number of boosting rounds is large.

d) eXtreme Gradient Boosting (XGBoost): eXtreme Gradient Boosting (XGBoost) is a scalable and highly optimised Boosting algorithm developed by Chen and Guestrin in 2016 [19]. It is an advanced implementation of the gradient boosting framework, in which each new tree is trained to predict the residual errors (i.e., the difference between the actual values and the current ensemble's predictions) of the previous trees. XGBoost extends standard gradient boosting with several key innovations: built-in L1 and L2 regularisation to penalise model complexity and prevent overfitting, a parallelised tree-construction algorithm for faster training, support for distributed computing, and an internal mechanism for handling missing values. The algorithm also employs tree pruning, removing branches that do not improve predictive accuracy, which further controls model complexity.

XGBoost rapidly gained widespread recognition for its exceptional performance on structured and tabular data, and it has powered numerous winning solutions in machine learning competitions [19]. Its regularisation capabilities make it particularly robust against overfitting, and its ability to handle missing values internally reduces the preprocessing burden. XGBoost is versatile and performs well across both classification and regression tasks.

A notable limitation of XGBoost is its training speed on very large datasets, where it can be considerably slower than more recent alternatives such as LightGBM. It also requires careful hyperparameter tuning to achieve optimal performance, and it does not natively handle categorical features, requiring manual encoding as a preprocessing step [19].

e) CatBoost: CatBoost (Categorical Boosting) is a Boosting algorithm developed by Prokhorenkova et al. at Yandex in 2018 [20]. Its distinguishing feature is its native support for categorical features, which are variables that represent discrete categories (e.g., country names or device types) rather than continuous numerical values. Most gradient boosting algorithms require categorical features to be manually encoded into numerical formats (e.g., via one-hot encoding) before training. CatBoost eliminates this requirement by using a technique called ordered target encoding, which encodes categorical values using statistics computed on the target variable while preserving the ordering of training samples to prevent target leakage — a form of data contamination that can result in overly optimistic model performance.

The primary advantage of CatBoost is its ability to handle categorical features directly and robustly, making it particularly well-suited to real-world datasets where such features are prevalent. It also tends to require less hyperparameter tuning

than XGBoost and LightGBM, and is robust to overfitting on small datasets. The main limitation of CatBoost is that its training time can be longer than LightGBM, particularly on large datasets, though its prediction speed is among the fastest of the three gradient boosting libraries [20].

f) Light Gradient Boosting Machine (LightGBM): Light Gradient Boosting Machine (LightGBM) is a highly efficient Boosting algorithm developed by Ke et al. at Microsoft in 2017 [21]. It was designed to address the computational bottlenecks of earlier gradient boosting methods, particularly their slow training speeds and high memory consumption on large datasets. LightGBM introduces two key algorithmic innovations. First, it uses a histogram-based learning algorithm, which discretises continuous feature values into a fixed number of bins, replacing the expensive per-feature sorting required by traditional gradient boosting with far cheaper histogram operations. Second, LightGBM employs a leaf-wise (best-first) tree growth strategy: rather than expanding all nodes at a given depth level simultaneously (as in XGBoost), it always splits the leaf that yields the greatest reduction in loss. This produces deeper, more complex trees with fewer nodes and can lead to higher accuracy.

Two additional optimisations further improve LightGBM's efficiency. Gradient-based One-Side Sampling (GOSS) retains all training instances with large gradients (which carry the most information) while randomly discarding a fraction of those with small gradients, reducing the number of data points used at each iteration without significantly compromising accuracy. Exclusive Feature Bundling (EFB) groups mutually exclusive sparse features into bundles, reducing the effective number of features and further accelerating training [21].

The result is that LightGBM trains significantly faster and uses less memory than XGBoost, making it the preferred choice when working with very large datasets or resource-constrained environments. It also supports categorical features natively, though less sophisticatedly than CatBoost. The primary limitation of LightGBM's leaf-wise growth strategy is a higher susceptibility to overfitting compared to level-wise methods, particularly on small datasets, and this requires careful tuning of depth and regularisation parameters [21].

III. Literature Review

DDoS attack detection and prevention have been an important issue in research literature ever since the SDN architecture paradigm was conceived. Many researchers have suggested ways of dealing with DDoS attacks. According to the authors Su et al. [2], there are two broad categories under which DDoS attacks can be categorized for detection and prevention techniques, namely statistical approaches and machine/deep learning approaches.

A. Statistical Analysis-Based DDoS Detection Approaches

Methods involving statistical analysis focus on the probability distribution of network traffic in order to recognize any unusual behaviour patterns [2], [22]. Typically, under normal circumstances, traffic patterns remain predictable, while DDoS attacks usually disrupt them to some extent.

Many statistical methods use information entropy as the primary indicator. Entropy was first proposed by Claude Shannon back in 1948 as a way of measuring the randomness or predictability of a certain dataset. In the context of network traffic, entropy describes the extent to which traffic flow remains predictable or varied. The higher the entropy value, the less predictable and concentrated the traffic flow is. This makes entropy an excellent trigger for DDoS attacks, as the latter significantly affect traffic patterns [23].

Varalakshmi and Thenmozhi [4] introduce Entropy Based DDoS Attack Detection and Mitigation Algorithm (EDDA) along with a stochastic mitigation system in SDN-IoT networks. The scheme uses the entropy values for source and destination IP address-port tuples for each time window. If the entropy value falls below a particular threshold, then the system generates rate limiting commands by sending an appropriate OpenFlow FlowMod message to the switches. When evaluated on a simulation environment containing 70 hosts and 9 switches with BoT-IoT data comprising of TCP, UDP, and ICMP SYN Flood attacks, the algorithm yields an accuracy of 97.2 percent, surpassing other similar baselines such as RMPSOM, DOCUSS, and FSOMDM. Nevertheless, the false positive rate shoots up to 72 percent for larger window sizes. Besides, the stochastic mitigation component of the system lacks sound reasoning behind the probability allocation mechanism for each switch, not to mention that the fixed entropy threshold cannot generalize to diverse IoT scenarios.

Tsobdjou, Pierre, and Quintero [7] address a principal shortcoming of entropy techniques which is their lack of adaptability to changes in legitimate traffic as resulting of using a fixed threshold value. They propose an online, five-component entropy-based detection system for client-server architectures. The crucial feature of the authors' approach is the application of the dynamic threshold algorithm that is derived on the theory proposed by Chebyshev. This allows the detection boundaries to adapt to evolving baseline traffic patterns, as opposed to being static. Tested on simulations done within GNS3 and on the CICIDS2017 dataset, the authors' system reaches perfect 100% detection rates and 0% false positives for both ICMP floods and SYN floods. Despite such promising results, there are several considerations to be made here: Firstly, the system assumes the pre-filtering of spoofed IP addresses in advance, which might not be the case in some cases; the scope of this study is relatively limited; and finally, the detection time of 7.89 seconds, although possibly acceptable in some settings, is likely problematic for higher throughput connections. The authors mention not testing their system for low-rate DDoS attacks, a common limitation of entropy-based techniques.

Tian and Miyata [24] address yet another issue associated with the use of entropy, which is that it fails to distinguish attacks from traffic peaks or flashes crowds.To solve this problem, the authors introduce a technique to detect anomalies

in SDN using a conditional entropy approach. Instead of analysing the entropy measure alone, this technique measures the entropy of destination IP addresses along with the conditional entropies of source port and packet sizes and sets the threshold values using normal traffic data. This technique succeeds in recognizing attack conditions from traffic peaks that single-measure methods such as one adopted by Varalakshmi and Thenmozhi [4] fail to identify. It attains an accuracy rate of 97.2% in 0.74 seconds. The downside is that it is only tested using a small simulated network environment with artificial traffic, limiting its applicability. Moreover, as the attacks are supposed to have larger volumes than normal traffic, it may be vulnerable to low-rate attacks and does not include any mitigation technique.

Stepping away from entropy altogether, Swami, Dave and Ranga [5] propose an Interquartile Range (IQR)-based method implemented in the Ryu controller. Thier system specifically targets spoofed SYN flooding attacks. This system monitors incoming `packet_in` messages and flags hosts whose SYN request rates exceed a threshold derived from Tukey's IQR rule [25], subsequently blocking confirmed attackers at the switch level via OpenFlow `FlowMod` messages. For single-attacker scenarios, the method achieves 100% detection accuracy; for multiple concurrent attackers, accuracy falls to 93%–95%. The approach also remarkably reduces controller load and `packet_in` traffic volume. Its scope, however, is narrow as the system is designed exclusively for spoofed SYN floods and has not been evaluated against UDP or ICMP flood variants, which account for a significant share of real-world DDoS activity. The experimental set-up, which is a single switch and 21 hosts, raises obvious scalability concerns, and detection delays of up to 157 seconds, coupled with a fixed 25-second detection interval, may be difficult to justify in time-sensitive operational environments.

Statistical methods, in general, provide an efficient and easily interpretable base for the DDoS attack detection in SDNs. They utilize traffic properties and, therefore, allow quick implementation and easy interpretation. Nevertheless, the reviewed articles demonstrate some common limitations of such solutions, including their vulnerability to threshold settings, failure to discriminate between attacks and peak traffic loads, as well as weak performance under multi-vector attacks. It is due to these drawbacks that machine learning methods are currently considered as more promising solutions.

B. Machine and Deep Learning-Based DDoS Detection Approaches

With the increasing complexity of DDoS attacks, researchers have turned their focus to developing adaptive approaches, with the advent of machine learning as a superior approach compared to rule-based approaches.

Garba et al. [10] propose a real-time SDN-based framework for DDoS attack detection and mitigation tailored to smart home IoT networks. Network traffic is continuously collected from OpenFlow switches using the RYU controller's flow-monitoring capability. For the SDN controller itself, a signature-based SNORT IDS is deployed with custom rules that monitor the controller's IP address and OpenFlow port; any matching attack traffic is immediately flagged. For the IoT devices, four lightweight supervised machine-learning models: Support Vector Machine, Logistic Regression, Decision Trees, and K-Nearest Neighbour, are trained on both normal and attack traffic captured from a real hardware testbed. After feature selection via Principal Component Analysis and correlation analysis, the best-performing model (Decision Tree) is exported and executed directly as a RYU controller application. Upon detection, the controller installs OpenFlow flow-mod rules to drop or reroute malicious flows from the attacking host. This design offers provides layered protection: SNORT safeguards the controller against control-plane exhaustion, while the ML models secure the heterogeneous IoT devices without requiring per-device rules. Furthermore, all detection and mitigation run in real time on the controller itself, preserving legitimate traffic with minimal interruption. Finally, the solution is low-cost and scalable, needing only a single controller application and no additional hardware. Their framework, however, suffers from several notable limitations.The authors only generated and evaluated TCP SYN flood attacks on their hardware IoT setup. Other common DDoS variants (UDP flood,reflective/amplification, slow-rate, or application-layer attacks) were not tested in the actual smart-home environment—only in the larger public datasets. This leaves uncertainty about how well the system would perform against the full spectrum of real-world threats. As author acknowledge, SNORT relies entirely on predefined rules for the controller's IP and OpenFlow port. Any new or obfuscated controller-targeted attack that does not match the existing signatures will bypass this layer completely.

Hnamte et al. [8] propose a DDoS attack detection and mitigation framework using a Deep Neural Network (DNN) architecture capable of integration with the Ryu SDN controller. They trained a DNN with 3 layers made up of 128, 256, and 128 neurons per layer, respectively. The model is trained and evaluated on InSDN, KaggleDDoS, and CICIDS2018, achieving remarkable accuracy of 99.98%, 99.99%, and 100%, respectively. Despite these remarkable results, the authors did not clearly provide the network topology (only describing what devices they used) on which the model was deployed, leaving questions about real-time performance. Furthermore, the authors mention the use of a threshold against which the classification output of the model is compared to quantify it as an attack or benign traffic. If this is fixed, it could limit the model's adaptivity.

Alashhab et al. [9] propose an ensemble online machine-learning model for real-time DDoS detection and mitigation in SDN environments. The system works as follows. A traffic collector module captures flow statistics from OpenFlow switches via the Ryu controller. These flows are pre-processed and streamed to an OML-based IDS that continuously trains and updates an ensemble of four online classifiers—BernoulliNB, Passive-Aggressive, SGD, and MLP—using incremental learning. When an attack is detected, the OML-based IPS immedi-

ately applies mitigation actions such as flow-rule installation to drop or reroute malicious traffic. The model adapts on-the-fly to new attack patterns without full retraining, and the entire pipeline runs in real time within the SDN controller. The short coming of this system is that authors rely on a static subset of 14 features chosen via a Chi-squared-based method. If new protocols, applications, or attack variants introduce previously unseen characteristics, the model may require manual re-engineering of the feature space rather than fully autonomous adaptation. Once an attack is detected, the IPS applies basic flow-rule actions (primarily drop or reroute). There is no advanced per-flow rate limiting, source trace–back, quarantine, or coordination across multiple controllers/domains, which could be necessary in large-scale or multi-tenant SDN deployments.

Pandey and Mishra [6] proposes an intelligent two-stage hybrid model for detecting DDoS attacks in IoT networks by incorporating entropy and ensemble techniques. The first stage of the model is where the heavy lifting happens. Four carefully selected classifiers; CatBoost, Random Forest, Extra Trees, and LGBM, each independently analyse the enriched feature set, which includes the original traffic features alongside computed entropy and conditional entropy values. The second stage combines decisions from these based models into a final decision. A logistic regression meta-classifier takes the predictions from all four Stage 1 classifiers as its input and learns how to best combine them into one reliable output. Rather than simply voting, it essentially learns which classifiers to trust more under which conditions. The models are trained and evaluated of CICIoT2023 achieving final accuracy of 99.86%. Despite the promising results, several limitations can be inferred from the proposed approach. The evaluation relies only on the CICIoT2023 dataset, which, while recent and large-scale, represents a single controlled IoT topology, raising concerns about the model's generalizability to real-world networks with diverse device configurations and evolving attack patterns. While the authors briefly acknowledge the emerging threat of adversarially crafted attacks, no empirical evaluation of the model's robustness against such evasion strategies is conducted, leaving a significant gap between laboratory performance and real-world resilience. Lastly, the experimental analysis is solely based on accuracy, which is not representative of performance when dealing with class imbalanced datasets.

Gebremeskel et al. [26] propose another hybrid model that comprise of entropy-based detector and deep LSTM module. When a large volume of packets arrives with the same repeated IP address, the switch detects rule mismatches and forwards these packets to the controller. The controller then computes the entropy value reported by the switches. This entropy is calculated based on a predefined threshold compared against the rule set for each suspicious host. Once the entropy exceeds the threshold, the controller identifies the occurrence of a DDoS attack. At the same time, the controller redirects packets associated with high entropy values to the deep neural network-based detection server module for further analysis, attack confirmation, and classification. Their proposed method is deployed in multi-controller environment and the detection and classification results are subsequently shared with neighbouring controllers in the distributed architecture. This hybrid deployment offer several benefits. First, entropy calculation (based on a threshold against the rule set per suspicious host) is computationally simple and efficient. It allows quick identification of potential DDoS attacks, especially flooding attacks that concentrate traffic on one or few destinations. Second, the two-stage design (lightweight entropy-based screening followed by deep detection server for confirmation and classification) avoids overloading the controller or network with heavy analysis on all traffic. Third, the multi-controller implementation solved single-point of failure in single-controller environments. Last, sharing results with neighbouring controllers in a distributed set-up adds collaboration and scalability, which helps in protecting the overall SDN architecture early. Despite these remarkable strengths, the detection heavily depends on analyzing only the first 50 incoming packets which can lead to high false positives during legitimate traffic bursts as well as delayed detection for slow-rate or sophisticated DDoS attacks that do not trigger obvious rule mismatches or entropy drops within the first 50 packets.

Wang and Wang [27] propose a lightweight online defence system for DDoS attacks in SDN that combines a hybrid CNN-ELM(Convolutional Neural Network and Extreme Learning Machine) detection model with an IP trace-back-based mitigation module. The controller collects `Packet_In` messages and flow table statistics from the switches. These are passed to the CNN-ELM module, where the convolutional neural network automatically extracts deep features from the traffic, and the extreme learning machine performs fast binary classification (normal or attack). Suspicious flows are logged in a compact blacklist. The mitigation module then applies an improved IP trace-back technique that uses ten packet attributes, including source and destination IP, MAC addresses, switch ID, ingress port, and crucially the time stamp.

This design offers several advantages. Firstly, the CNN-ELM hybrid achieves excellent detection accuracy while keeping computational cost low, because the ELM classifier trains in a single step without iterative back-propagation. Secondly, feature construction is efficient: the model uses a small set of twelve flow-table features plus four simple statistical features that capture DDoS-specific patterns. Thirdly, the IP trace-back method enables precise source identification and immediate blocking, rather than just dropping traffic at the victim. Finally, the entire system runs online in real time and was shown to restore normal traffic within seconds in Mininet experiments. Despite these strengths, the method is fully supervised and therefore depends on large amounts of labelled training data, which can be costly and difficult to obtain for new attack variants in real-world SDN deployments.

Najar and Naik [28] propose an efficient CNN-based detection and mitigation framework (BRS + CNN) for DDoS attacks in SDN environments. Network flows are collected from OpenFlow switches via the POX controller. After balanced random sampling and feature selection on the CICDoS2019

dataset, a 1D-CNN model is trained for both binary (normal vs. attack) and multi-class classification. When an attack is detected, the controller immediately applies mitigation: traffic filtering, rate limiting, and ip-tables rules to block spoofed IPs. A dedicated monitoring module continuously checks blocked IPs and removes them from the blacklist once the attack ceases, ensuring legitimate traffic is not permanently dropped. An email notification system instantly sends detailed attack information (source IP, blocked IPs, bandwidth usage, and time stamp) to the administrator. This design offers several clear advantages. First, the balanced random sampling and careful feature selection produce very high accuracy (99.99% binary, 98.64% multi-class) with remarkably low computational cost and fast inference. Second, the mitigation layer combines filtering, rate limiting, and dynamic IP blocking, while the monitoring system prevents over-blocking of legitimate hosts. Third, the inbuilt email notification provides administrators with immediate, actionable context without requiring manual log checking. In spite of these strengths, the framework was evaluated only on the CICDoS2019 dataset and a single simulated topology in Mininet/POX; its performance in large-scale, heterogeneous, or production SDN environments with dynamic traffic patterns remains untested.

Overall, there seems to be an obvious trend within the aforementioned methods, ranging from conventional entropy-based models, to simple yet less comprehensive single-classifiers, ensemble models, to hybrid model comprising both entropy and deep learning frameworks.

IV. Methodology

With a comprehensive description of the ensemble models considered in this study provided in the previous section, this section presents a detailed description of the proposed model evaluation and comparison scheme.

A. Custom Topology Description

Although [12] presents the latest IoT dataset for modern sophisticated DDoS attacks, it was created on a different network topology, making it difficult to rely upon for generalisation to alternative topologies. As a result, this work implements a custom IoT network topology, as illustrated in Fig. 2. The network consists of two aggregation switches, each connected to five IoT devices. These devices may represent any class of smart sensor, such as cameras or temperature sensors, connected to the network. A core switch links the aggregation layer directly to a remote server, which is responsible for collecting and analysing data from the sensors to perform arbitrary sensor-based decision-making. Additionally, an attacker is assumed to have maliciously connected to the network and is capable of recruiting IoT nodes to launch a large-scale DDoS attack, overwhelming the system with redundant packets. This kind of architecture is common in automotive systems, smart cities, and smart agriculture applications. The switches are controlled by a POX controller, which leverages OpenFlow to install flow table rules and collect flow statistics. The network was implemented using the Mininet SDN emulation framework, with Python used to define the POX controller logic. A new dataset comprising the same feature set as the CICIoT-DIAD2024 dataset [12] was subsequently generated, upon which the models were evaluated and compared.

B. CICIoT-DIAD2024 Dataset Description

The CICIoT-DIAD2024 is a comprehensive IoT dataset comprising 33 distinct attacks conducted on an IoT topology of 105 devices at the Canadian Institute for Cybersecurity [12]. The dataset includes 134 behaviour-based features and 84 flow-based features. This study employs the flow-based feature set, as it captures more granular network patterns at the packet level, consistent with common practice in the literature [11], [28]. The DDoS attack subset contains 12 distinct attack types; however, only four were available in CSV format at the time of writing. These are: ACK Fragmentation, ICMP Flood, ICMP Fragmentation, and HTTP Flood, and are therefore the only classes considered in this study.

C. Custom Dataset Generation Procedure

The custom dataset was generated by operating the SDN controller in a dedicated data-collection mode during controlled traffic simulations. Each traffic type, including benign sensor traffic and each DDoS attack variant, was run independently for a period of 20 minutes, during which the controller continuously extracted and logged flow-level features at each polling interval of 10 seconds.

Feature vectors were reconstructed within dedicated `_reconstruct_features` python function, which derives CICFlowMeter-compatible features from the limited statistics exposed by OpenFlow 1.0(POX uses this OpenFlow version). Since OpenFlow provides only cumulative packet counts, byte counts, and flow duration per flow entry, the reconstruction procedure approximates the full feature set through the following steps.

At each polling interval, the controller requests flow table statistics from all connected switches via `ofp_stats_request` messages. For each flow identified by a (source IP, destination IP) tuple, the cumulative packet and byte counts reported by the switch are compared against those recorded in the previous polling cycle to compute per-interval deltas. These deltas, along with the elapsed wall-clock time between polls, are appended to rolling history lists maintained per flow, retaining at most the 50 most recent intervals.

From these accumulated records, rate-based features are derived first. Flow Bytes/s and Flow Packets/s are computed by dividing the total cumulative byte and packet counts by the flow duration. Forward packet rate is approximated under the simplifying assumption that, from the perspective of each source host, all observed traffic is unidirectional (forward). Packet length statistics, including mean, standard deviation, minimum, maximum, and variance, are estimated by computing the byte-to-packet ratio within each polling interval and then deriving descriptive statistics across all recorded intervals. The average packet size is set equal to the mean packet length.

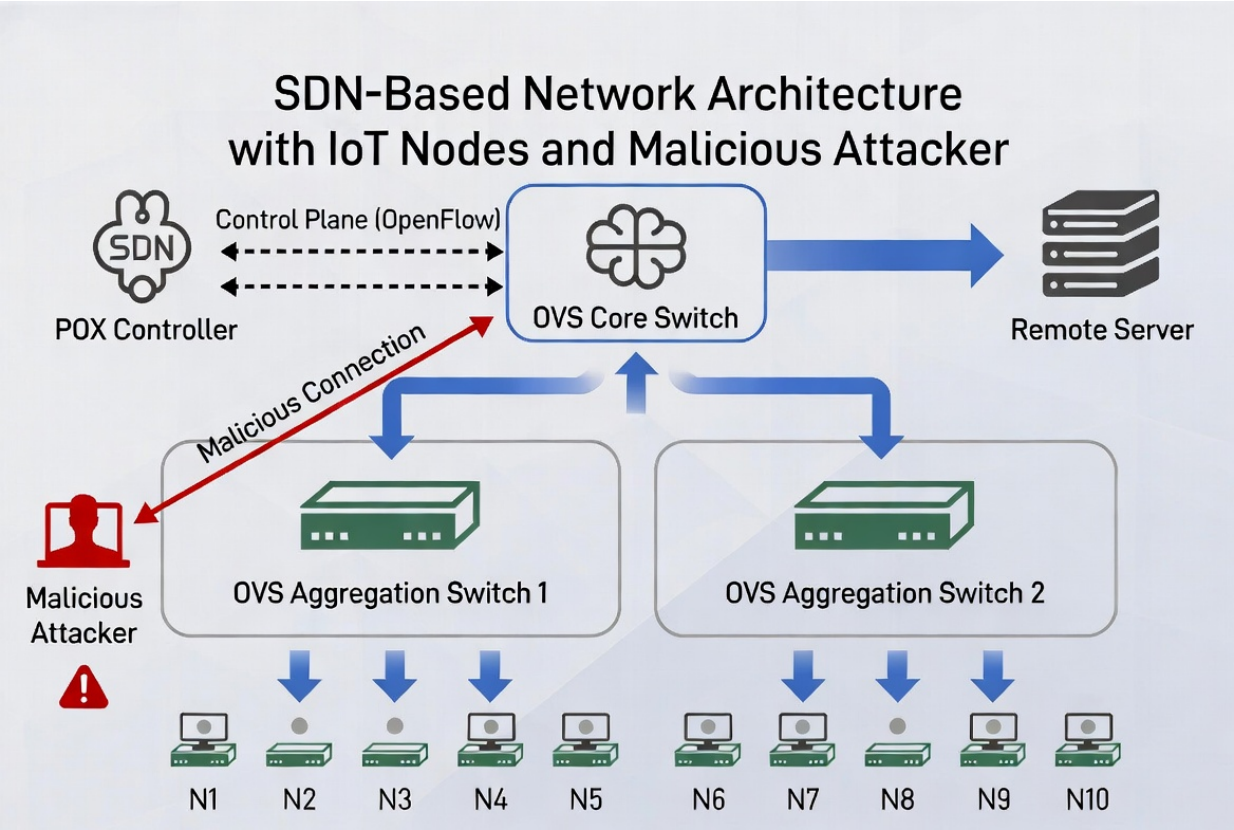

Fig. 2: SDN-Based IoT Network Topology

Inter-arrival time (IAT) features are derived from the delta time list, which records the elapsed time between successive polling events for a given flow. The mean, standard deviation, maximum, and minimum of these intervals serve as proxies for flow-level and forward-direction IAT statistics. Backward IAT features, which require per-direction flow visibility not available in OpenFlow 1.0, are set to zero throughout.

TCP flag counts, comprising SYN, ACK, FIN, RST, PSH, and URG flags, are accumulated separately via `PacketIn` events. Since `PacketIn` messages are triggered only for packets that do not match an existing flow rule (i.e., the first packet of each new micro-flow), the flag counters reflect early-stage TCP handshake and control activity rather than the full traffic volume. Header length is approximated by multiplying the total packet count by a fixed 20-byte IP header size. Subflow packet and byte counts are set equal to the total forward counts, and the initial TCP window size in the forward direction is approximated as the minimum of the total byte count and the maximum TCP window value of 65,535 bytes. Features that cannot be reconstructed from OpenFlow statistics — including all backward-direction lengths, bulk transfer metrics, and idle time statistics — are explicitly set to zero.

Once assembled, the feature dictionary is aligned to the column ordering of the CICIoT-DIAD2024 dataset and appended as a new row to a traffic-type-specific CSV file, with a corresponding class label. This process repeats at every polling interval for the duration of each simulation run, yielding a labelled dataset whose feature schema is directly comparable to that of the CICIoT-DIAD2024 dataset.

In this new dataset, HTTP FLOOD was replaced with generation of dataset for UDP FLOOD, so as to have only network layer attacks to analyse.

D. Model Comparison Scheme

For the comparison of model performances, several schemes have been proposed in the literature. This research adopts a scheme similar to that employed by Saeid, Gurguis, and Madbouly [11], as illustrated in Fig. 3.

The scheme consists of four primary stages. The first stage is data preprocessing, an essential step in preparing raw data for machine learning algorithms. It involves cleaning, transforming, and reformatting the data to render it suitable for model training. The quality of this stage can have a significant impact on downstream model performance. The second stage involves dataset balancing, shuffling, and partitioning into two subsets: a training set and a test set, using a 70/30 split ratio, as opposed to the 80/20 ratio adopted in [11]. The third stage covers model training and evaluation, where the training set is used to fit each model and the test set is used to assess generalisation performance on unseen data. The final stage involves a comparative analysis of all models based on a set of predefined evaluation metrics.

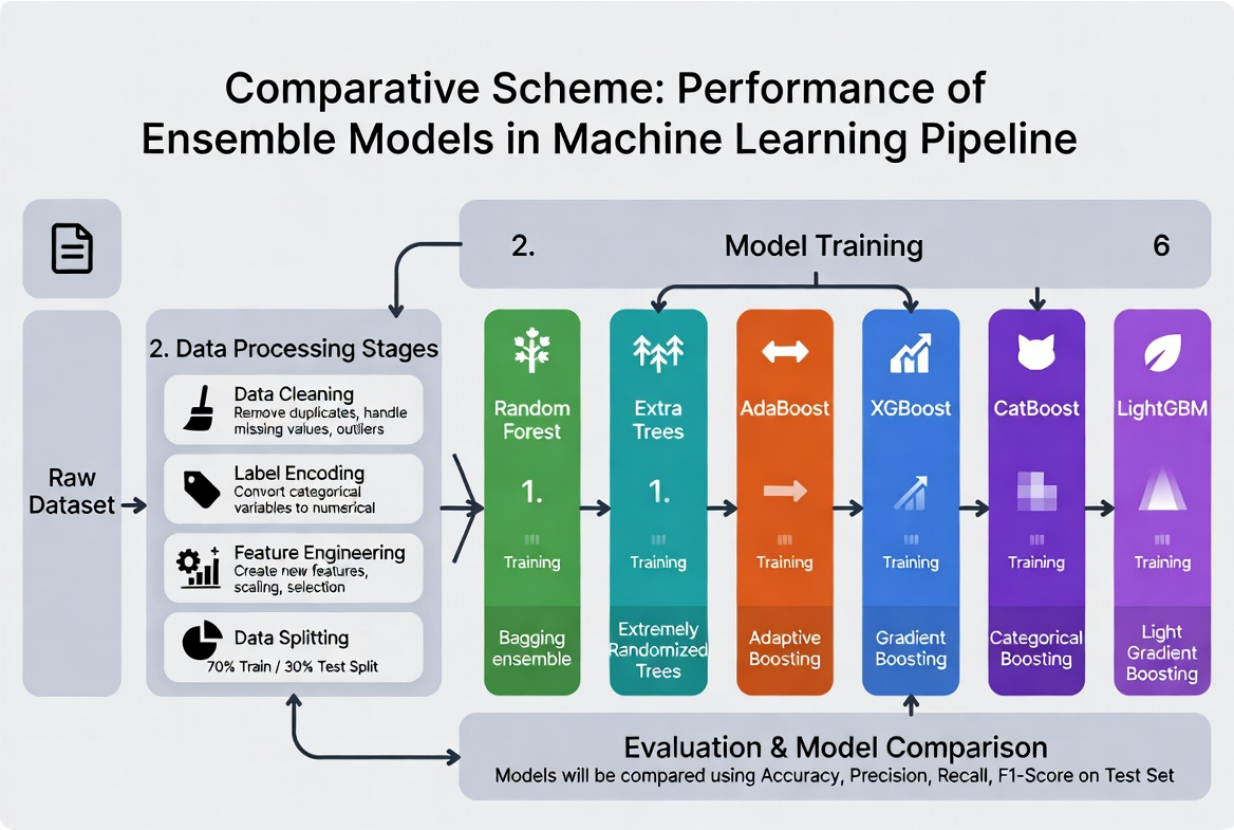

Fig. 3: Ensemble Model Comparison Scheme

E. Data Preprocessing Pipeline

The preprocessing stage consists of several sequential sub-steps, and was adopted directly form the work of Najar and Naik [28]. The raw data is first loaded from multiple CSV files — one per DDoS attack class — using the `pandas` library. Following ingestion, the data is inspected for missing or infinite values, which are subsequently removed as appropriate. Duplicate records are identified and discarded to prevent bias during training. Feature columns are then examined for constant or near-zero variance, and such uninformative features are dropped. While reference [28] adopts a 70% correlation threshold for feature elimination, this study employs a more stringent 95% threshold to better control redundancy among highly collinear variables. In high-dimensional datasets, moderately correlated features (e.g., 0.7) may still capture complementary information and contribute to model generalization. Eliminating such features prematurely can lead to information loss and reduced predictive performance. By contrast, a 95% threshold targets only near-duplicate features, ensuring that strongly redundant variables are removed while preserving those that may encode subtle but meaningful variations in the data. Categorical variables, are encoded numerically using label encoding. For tree-based models, features do not necessarily need to be scaled as it would with Neural Networks Architectures, as such no scaling was applied to the resulting features. Finally, the class labels are encoded and the dataset is balanced using resampling technique adopted from in [28] was used to address class imbalance prior to splitting.

F. Evaluation Metrics

Model performance is assessed using both Predictive Performance metrics (Accuracy, Precision, Recall, F1-Score) and Resource Utilization (CPU Utilisation and execution time at both training and inference time). Predictive performance metrics are widely adopted in literature [9], [10], [29], [30]. Accuracy measures the proportion of correctly classified instances out of all instances, providing a general indicator of model performance. Precision quantifies the fraction of positive predictions that are truly positive, reflecting the model's ability to avoid false alarms. Recall, also referred to as sensitivity, measures the proportion of actual positive instances that are correctly identified, capturing the model's ability to detect attacks. The F1-Score is the harmonic mean of Precision and Recall, providing a balanced measure particularly suited to imbalanced classification problems. CPU Utilization is percentage of CPUs used by process during its execution. Pythons's `psutil` was used to measure this metric.

V. Results and Discussions

This section presents an in-depth performance analysis of six ensemble learning algorithms: *Random Forest*, *CatBoost*, *Extra Trees*, *AdaBoost*, *LightGBM*, and *XGBoost* evaluated

across two distinct datasets. The first is a balanced *Custom-Dataset* comprising 14,978 samples which was partitioned with a 70:30 train-test split, and second is the large-scale *CICIoT-DIAD2024* benchmark containing 470,464 samples. Models are assessed based on predictive performance (Accuracy, Precision, Recall, F1-Score) and computational resource consumption (CPU training and inference utilisation, training time, and inference latency).

A. Predictive Performance Results

1) CustomDataset: Figure 4 shows the predictive performance of the models on the CustomDataset which was generated as described in section IV. Remarkably, all models exhibit extremely high and consistent predictive performance, with F1-Scores ranging between 99.41% (AdaBoost) and 99.77% (LightGBM), differing by just 0.36 percentage points. The best performer here is the LightGBM with the highest F1-Score of 99.76% and the highest Accuracy of 99.77%. As such, LightGBM can be considered the optimal model for this CustomDataset in terms of its raw prediction capability. The XGBoost is only slightly behind with an F1-Score of 99.69%, while the Random Forest and the CatBoost exhibit identical F1-Scores of 99.61%, indicating that they create highly similar decision surfaces. Finally, the ExtraTrees and the AdaBoost occupy the last two positions in terms of prediction performance, but only with slight gaps of 99.45% and 99.41% correspondingly, which seems to be insignificant at this level. In other words, the CustomDataset does not help us distinguish the models in terms of prediction performance.

2) CICIoT-DIAD2024: The CICIoT-DIAD2024 dataset(Fig 5) provides a far more challenging and informative test-bed. Here, the performance hierarchy becomes more pronounced, and the divergence between Precision and Recall surfaces clearly, indicating class imbalance or overlapping decision boundaries. XGBoost achieves the highest overall F1-Score of 96.13% and the highest Accuracy of 96.35%, narrowly surpassing Random Forest (F1: 96.16%, Accuracy: 96.01%) and LightGBM (F1: 95.79%, Accuracy: 95.95%). Notably, LightGBM achieves the highest Precision across all models at 96.64%, suggesting it is the most conservative classifier — it predicts positive labels only when it is highly confident, resulting in fewer false positives at the cost of a slightly lower Recall (95.29%). ExtraTrees also displays this bias, with a Precision of 96.30% paired with a Recall of 94.95%, the lowest recall among the top-tier models.

The CICIoT-DIAD2024 dataset(Fig 5) presents a much more challenging environment for testing. In particular, the gap between the algorithms' performance gets accentuated, and differences in Precision and Recall become evident, pointing to an unbalanced problem setting or overlapping classes. The model that demonstrates the best performance in terms of F1-Score of 96.13% and Accuracy of 96.35% is XGBoost. It is followed by Random Forest with scores of F1-Score of 96.16% and Accuracy of 96.01% and then by LightGBM with F1-Score of 95.79% and Accuracy of 95.95%. Of particular note is that LightGBM demonstrates the highest Precision score of 96.64%, indicating its conservativeness; that is, the model tends to predict positive labels only when it is very sure, which results in fewer false positives but some drop in Recall to 95.29%. ExtraTrees is another algorithm displaying such behaviour: Precision of 96.30% combined with the lowest Recall in this set of top-performing algorithms – 94.95%. This precision-recall trade-off is a meaningful practical distinction in IoT threat detection contexts, where the cost of false negatives (missed attacks) and false positives (unnecessary alerts) must be weighed.

CatBoost falls comfortably in the middle range with a score of 95.11%, doing well, but not particularly impressive. AdaBoost clearly under performs on this large-scale dataset, as is the case on customDataset, recording the lowest F1-Score of 93.21% and a Precision of only 93.34%. This is explained by a fundamental limitation of its boosting mechanism under conditions of large-scale and potentially noisy data: sequential weak learners amplify errors when base learners struggle with class complexity or overlapping feature distributions.

B. Computational Resource Utilisation

Efficiency of computation(Fig 6 is what distinguishes these models far better than predictive performance,especially when it comes to the CICIoT-DIAD2024 dataset. The training time of 458.73 seconds for AdaBoost on the large dataset is nothing less than a catastrophe compared to its competitors: it takes around 48 times longer to train than the Random Forest model (9.49 s), and 32 times longer to train than the ExtraTrees model (11.51 s). That is because of the sequential nature of boosting in the AdaBoost algorithm, which means that each weak learner has to be trained and tested separately, making parallelisation impossible. In addition, the CPU usage during the training process remains quite low at 17.35% ,indicating that the model does not utilize available resources effectively. It is simply slow by design.

Random Forest and ExtraTrees achieve the fastest training times on CICIoT-DIAD2024 at 9.49 s and 11.51 s respectively, benefiting from their inherently parallel tree construction. This comes at the cost of near-complete CPU saturation, 94.39% for Random Forest and 95.25% for ExtraTrees, which would be a concern in resource-constrained deployment environments. LightGBM (71.45 s) and XGBoost (62.98 s) fall in an intermediate zone, trading off training speed against more moderate CPU load than the parallel bag-based methods. CatBoost is particularly noteworthy for its efficiency on the CustomDataset (1.26 s training time) but scales poorly to the larger dataset (37.58 s), mainly due to its ordered boosting mechanism incurring greater overhead as sample count increases.

Latency at inference time can be considered the most essential parameter when working with real-time IoT security monitoring systems. The best results are obtained using the CatBoost algorithm, which yields inference times of 0.005 seconds for CustomDataset and an impressive 0.0078 seconds for CICIoT-DIAD2024, thereby outperforming all other models irrespective of the volume of data. This can be attributed to the usage of oblivious decision trees in CatBoost, which

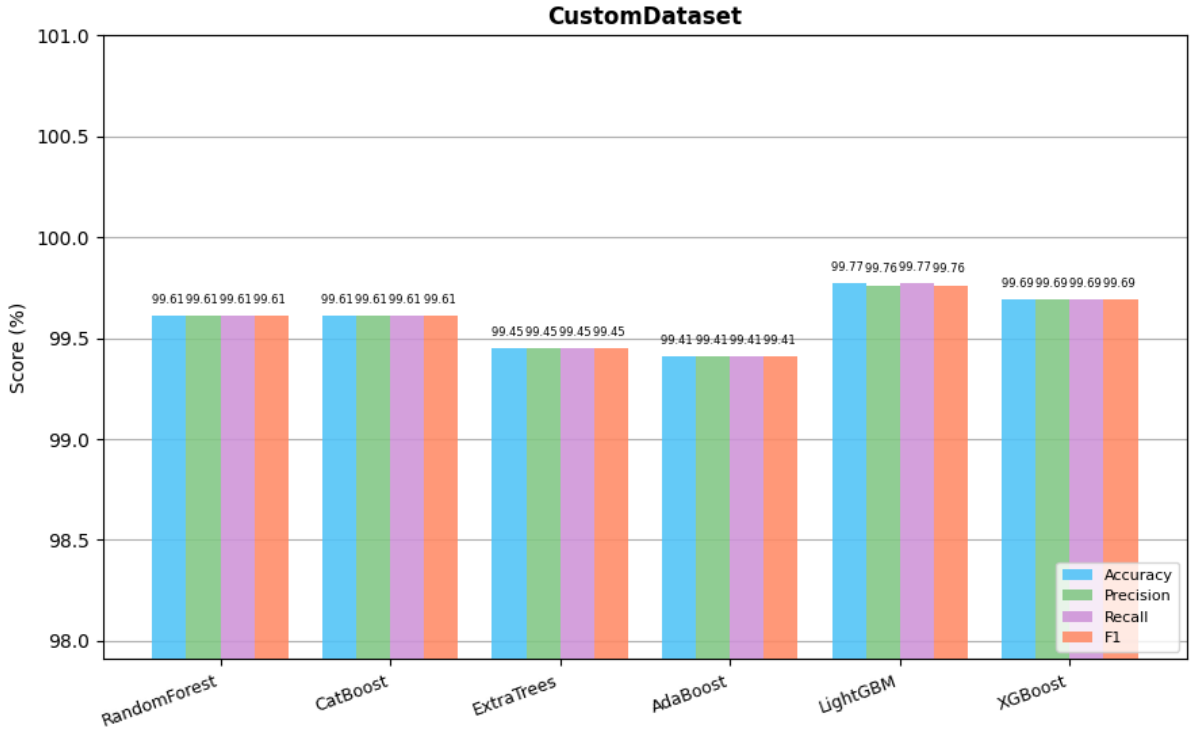

Fig. 4: Model's Predictive Performance on Custom Dataset

have a symmetrical tree architecture allowing predictions in vectors without moving through asymmetrical paths. Similarly, XGBoost has good inference times of 0.02 seconds on Custom and 0.98 seconds on CICIoT-DIAD, which is practical for production use.

LightGBM appears to be the worst performer, taking 3.45 s along with 88.23% CPU utilisation; making it the most resource-heavy model when it comes to inference. This is a non-trivial finding: LightGBM achieves the highest precision and is among the top-three F1 models, but its inference cost may disqualify it for latency-sensitive applications LightGBM is the most accurate and is one of the three best F1-performing models, but its inference performance might rule it out of use cases which are sensitive to inference latency. AdaBoost records the second-highest inference time (1.21 s on CICIoT), compounding its already poor training efficiency. The inference time of ExtraTrees (0.29 s) is acceptable, although its high CPU inference usage (59.75%) deserves attention.

C. Cross-Dataset Generalisation and Stability

A cross-dataset comparison(Fig. 7 reveals how each model scales with data volume and complexity. The F1-Score drop from CustomDataset to CICIoT-DIAD2024 is substantial for all models, reflecting the added complexity of the larger benchmark. AdaBoost experiences the largest drop (from 99.41% to 93.21%), losing 6.20 percentage points, which indicates that it does not generalise well to large, complex, and potentially imbalanced data. On the other hand, ExtraTrees suffers the second-largest drop (from 99.45% to 95.47%), likely attributable to its fully random feature splits introducing excess variance at scale. XGBoost exhibits the best generalisation stability, dropping from 99.69% to 96.13%. This is a reduction of 3.56 points while retaining the highest absolute F1 on CICIoT-DIAD2024. Random Forest follows closely with a drop of 3.45 points. CatBoost and LightGBM show moderate drops (4.50 and 4.00 points respectively), remaining competitive on both datasets.

VI. Conclusions and Future Work

This study was set to answer an important question: *among the widely used tree-based ensemble algorithms, which one is best suited for detecting IoT botnet DDoS attacks, and does that conclusion hold across different network environments?* To answer this, six models namely, Random Forest, Extra Trees, AdaBoost, XGBoost, LightGBM, and CatBoost, were evaluated under consistent conditions on two distinct datasets: a custom-generated SDN-IoT dataset and the large-scale CICIoT-DIAD2024 benchmark.

The results tell a clear story, though not a simple one. On the custom dataset, all six models performed remarkably well, with F1-scores clustered between 99.41% and 99.77%. LightGBM edged ahead with the highest F1-score of 99.76%, but the margins were too small to draw strong conclusions about which model is truly superior. The custom dataset, while useful for validating the experimental pipeline, was simply not challenging enough to separate the models. This is mainly due to simplification assumption made to transform

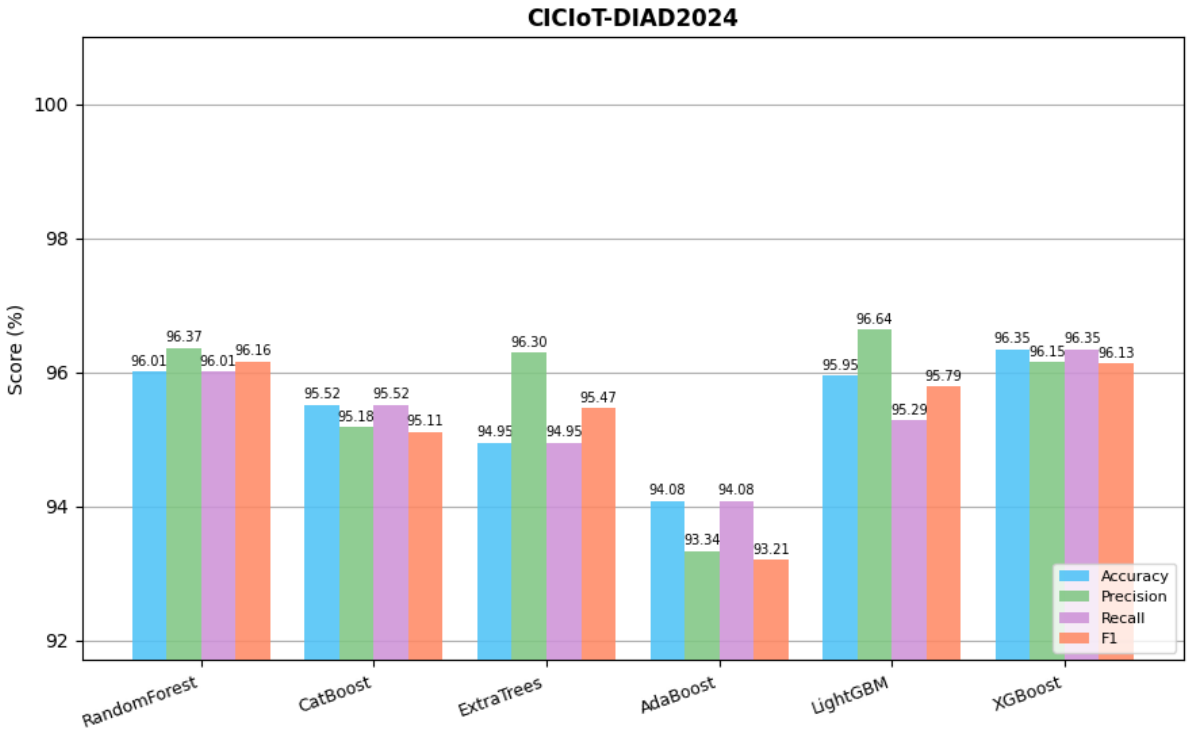

Fig. 5: Model's Predictive Performance on CICIoT-DIAD2024

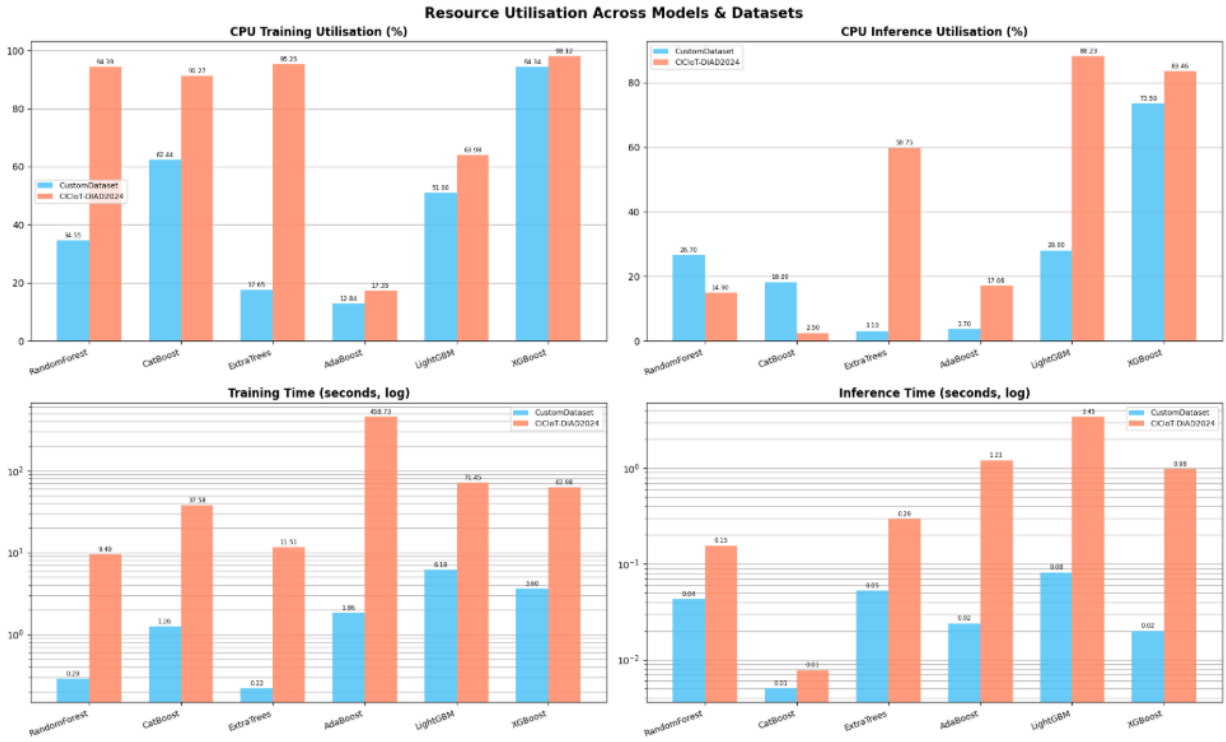

Fig. 6: Model's Computation Resource Utilization: Training & Inference Time and CPU Utilization

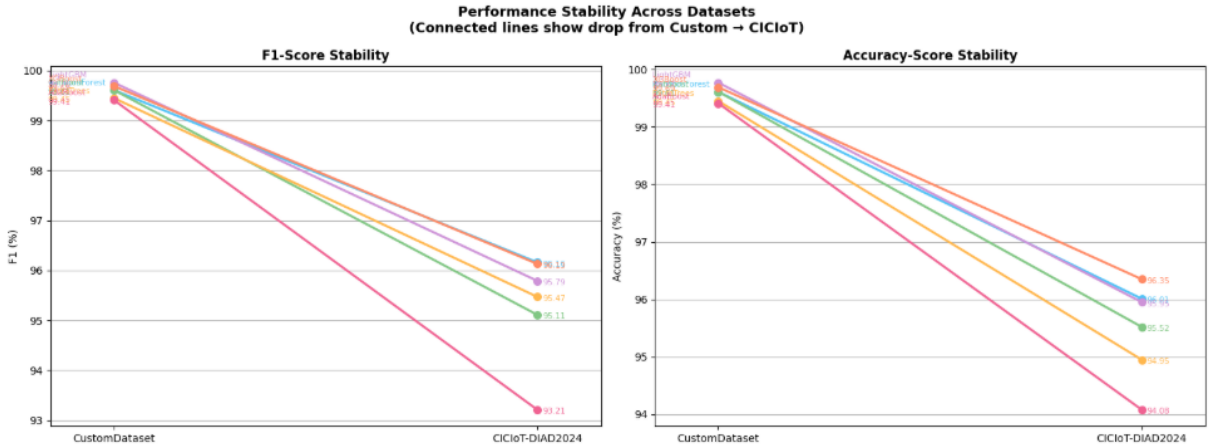

Fig. 7: Model's Cross-Dataset Performance Comparison

CICFlowMeter features to POX Controller-based features, which uses OpenFlow 1.0.

However, the CICIoT-DIAD2024 dataset proved far more informative. Here, the gaps between models were more vivid, and a clearer ranking was obvious. XGBoost gave the best overall accuracy at 96.35%, with an F1-score of 96.13%, while Random Forest followed closely with an F1-score of 96.16%. These two models consistently showed the strongest balance between detection performance and generalisation stability across both datasets. LightGBM achieved the highest precision of 96.64%, making it the most conservative classifier. This is useful when false alarms carry a high cost, but its inference time of 3.45 seconds and heavy CPU usage at inference make it a poor fit for real-time IoT monitoring. CatBoost, on the other hand, stood out for its inference speed, completing predictions in under 8 milliseconds regardless of dataset size, which makes suitably advantageous in latency-sensitive deployments.

Furthermore, AdaBoost was the clear underperformer across the board. Its training time of 458.73 seconds on the large dataset which is nearly 48 times slower than Random Forest, combined with the lowest F1-score of 93.21% and the largest cross-dataset performance drop of 6.20 points, makes it a poor choice for this problem setting. Its sequential boosting design simply does not scale well to large, complex, and imbalanced traffic data.

From a practical standpoint, no single model wins on every dimension. If raw detection performance and generalisation are the priority, XGBoost and Random Forest are the strongest candidates. If inference latency is the bottleneck, CatBoost is the clear choice. If training resources are constrained, Random Forest and Extra Trees train fastest, though they saturate CPU resources in doing so.

Several directions remain open for future work. First, this study used only four of the twelve available DDoS attack types in the CICIoT-DIAD2024 dataset due to data availability at the time of writing. Expanding the evaluation to cover all attack classes would give a more complete picture of each model's strengths and weaknesses. Second, the custom dataset feature reconstruction was limited by the constraints of OpenFlow 1.0, which meant that several backward-direction and bulk-transfer features had to be set to zero. Using a more expressive data collection method , such as CICFlowMeter running directly on the network, would produce richer and more accurate feature vectors. Third, all experiments were conducted in single-controller SDN environments. Evaluating these models in multi-controller topologies would better reflect the complexity of real-world deployments. Fourth, none of the models in this study were subjected to systematic hyperparameter optimisation; applying techniques such as grid search or Bayesian optimisation could push performance further, particularly on the harder benchmark.

Finally, and perhaps most importantly, this study focused entirely on detection. A natural and necessary next step is to close the loop by integrating the best-performing models directly into the SDN controller to perform active attack mitigation, automatically installing flow rules to block or rate-limit identified attack traffic in real time. Bridging the gap between detection and response is what would make these systems genuinely useful in operational IoT environments, and it remains the most impactful direction for future work.

References

[1] M. J. Santos-Neto, J. L. Bordim, E. A. Alchieri, and E. Ishikawa, "Ddos attack detection in sdn: Enhancing entropy-based detection with machine learning," *Concurrency and Computation: Practice and Experience*, vol. 36, no. 11, p. e8021, 2024.

[2] Y. Su, D. Xiong, K. Qian, and Y. Wang, "A comprehensive survey of distributed denial of service detection and mitigation technologies in software-defined network," *Electronics*, vol. 13, no. 4, p. 807, 2024.

[3] Q. Waseem, W. I. S. W. Din, A. Aminuddin, M. H. Mohammed, and R. F. A. Aziza, "Software-defined networking (sdn): a review," in *2022 5th international conference on information and communications technology (ICOIACT)*. IEEE, 2022, pp. 30–35.

[4] I. Varalakshmi and M. Thenmozhi, "Entropy based earlier detection and mitigation of ddos attack using stochastic method in sdn_iot," *Measurement: Sensors*, vol. 39, p. 101873, 2025.

[5] R. Swami, M. Dave, and V. Ranga, "Iqr-based approach for ddos detection and mitigation in sdn," *Defence Technology*, vol. 25, pp. 76–87, 2023.

[6] N. Pandey and P. K. Mishra, "Conditional entropy-based hybrid ddos detection model for iot networks," *Computers & Security*, vol. 150, p. 104199, 2025.

[7] L. D. Tsobdjou, S. Pierre, and A. Quintero, "An online entropy-based ddos flooding attack detection system with dynamic threshold," *IEEE Transactions on Network and Service Management*, vol. 19, no. 2, pp. 1679–1689, 2022.

[8] N. T. H. NHUNG, V. Hnamte, A. A. Najar, J. Hussain, and M. Naik, "Ddos attack detection and mitigation using deep neural network in sdn environment," 2024.

[9] A. A. Alashhab, M. S. Zahid, B. Isyaku, A. A. Elnour, W. Nagmeldin, A. Abdelmaboud, T. A. A. Abdullah, and U. D. Maiwada, "Enhancing ddos attack detection and mitigation in sdn using an ensemble online machine learning model," *IEEE access*, vol. 12, pp. 51 630–51 649, 2024.

[10] U. H. Garba, A. N. Toosi, M. F. Pasha, and S. Khan, "Sdn-based detection and mitigation of ddos attacks on smart homes," *Computer Communications*, vol. 221, pp. 29–41, 2024.

[11] M. Saied, S. Guirguis, and M. Madbouly, "A comparative analysis of using ensemble trees for botnet detection and classification in iot," *Scientific Reports*, vol. 13, no. 1, p. 21632, 2023.

[12] M. Rabbani, J. Gui, F. Nejati, Z. Zhou, A. Kaniyamattam, M. Mirani, G. Piya, I. Opushnyev, R. Lu, and A. A. Ghorbani, "Device identification and anomaly detection in iot environments," *IEEE Internet of Things Journal*, dec 2024.

[13] N. Feamster, H. Balakrishnan, J. Rexford, A. Shaikh, and J. Van Der Merwe, "The case for separating routing from routers," in *Proceedings of the ACM SIGCOMM workshop on Future directions in network architecture*, 2004, pp. 5–12.

[14] A. Greenberg, G. Hjalmtysson, D. A. Maltz, A. Myers, J. Rexford, G. Xie, H. Yan, J. Zhan, and H. Zhang, "A clean slate 4d approach to network control and management," *ACM SIGCOMM Computer Communication Review*, vol. 35, no. 5, pp. 41–54, 2005.

[15] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[16] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, pp. 3–42, 2006.

[17] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[18] R. E. Schapire, "Explaining AdaBoost," in *Empirical Inference*, B. Schölkopf, Z. Luo, and V. Vovk, Eds. Springer, 2013, pp. 37–52.

[19] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[20] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, vol. 31, 2018, pp. 6638–6648.

[21] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.

[22] C. Singh and A. K. Jain, "A comprehensive survey on ddos attacks detection & mitigation in sdn-iot network," *e-Prime-Advances in Electrical Engineering, Electronics and Energy*, vol. 8, p. 100543, 2024.

[23] L. Li, J. Zhou, and N. Xiao, "Ddos attack detection algorithms based on entropy computing," in *International Conference on Information and Communications Security*. Springer, 2007, pp. 452–466.

[24] Q. Tian and S. Miyata, "A ddos attack detection method using conditional entropy based on sdn traffic," *IoT*, vol. 4, no. 2, pp. 95–111, 2023.

[25] A. Kolbaşı and A. Ünsal, "A comparison of the outlier detecting methods: An application on turkish foreign trade data," *Journal of Mathematical Sciences*, vol. 5, pp. 213–234, 08 2021.

[26] T. G. Gebremeskel, K. A. Gemeda, T. G. Krishna, and P. J. Ramulu, "Ddos attack detection and classification using hybrid model for multi-controller sdn," *Wireless Communications and Mobile Computing*, vol. 2023, no. 1, p. 9965945, 2023.

[27] J. Wang and L. Wang, "Sdn-defend: a lightweight online attack detection and mitigation system for ddos attacks in sdn," *Sensors*, vol. 22, no. 21, p. 8287, 2022.

[28] A. A. Najar and S. M. Naik, "Cyber-secure sdn: A cnn-based approach for efficient detection and mitigation of ddos attacks," *Computers & Security*, vol. 139, p. 103716, 2024.

[29] G. Fioravanti, M. G. Spina, and F. De Rango, "Entropy based ddos detection in software defined networks," in *2023 IEEE 20th Consumer Communications & Networking Conference (CCNC)*. IEEE, 2023, pp. 636–639.

[30] W. I. Khedr, A. E. Gouda, and E. R. Mohamed, "Fmdadm: A multi-layer ddos attack detection and mitigation framework using machine learning for stateful sdn-based iot networks," *Ieee Access*, vol. 11, pp. 28 934–28 954, 2023.